

分子對接模擬系統—以 HADOOP 平台及 CUDA 架構實作

李育郝, 蔡林鋒, 許弘駿

醫學資訊學系

慈濟大學

{99325108, 99325103}@stmail.tcu.edu.tw, hchsu@mail.tcu.edu.tw

摘要

本論文提出一套 MDSH(Molecular Docking Simulation based on Hadoop and CUDA)系統, 使用 Hadoop 平行架構及 GPU 運算單元, 以基因演算法搭配蒙特卡羅搜尋演算法進行生物分子對接模擬, 可有效降低模擬所需時間, 並提升藥物開發效率, 透過 Hadoop 架設雲端平台, 可減少研究人員在硬體設備上投資之成本。

Index Terms— *Hadoop, GPU, CUDA, GA, Monte-Carlo, Molecular Dynamic, drug design.*

1. 簡介

蛋白質是一種複雜的有機化合物, 也是生物體最重要的組成成分, 參與細胞每一個活動過程。通常蛋白質與產生反應的配體有高度的專一性, 而專一性受到許多影響, 如活性位點、受質點、形狀結構、表面電荷及親水性等等。在藥物開發上透過研究蛋白質與各式各樣化合物分子對接互動的結果, 找出能夠進行修復、促進、抑制或是破壞等作用的化合物分子, 並將此分子進行分析研究以及實驗。

模擬蛋白質與化合物分子的對接能夠輔助藥物開發[1], 分子動力對接模擬是將蛋白質定為受體, 把與目標受體對接的藥物分子定為配體, 使用電子計算機根據現實中的力場函式來評估兩分子之間所受到的作用力數值, 並分析兩者對接互動的結果, 用此數據來評斷此分子作為藥物的潛力。如此, 便可透過模擬分子對接的方式找出最佳結合模式。

對接分為柔性與剛性兩種方法[2], 剛性方法只考慮外表形狀的嵌合性, 可快速的初步篩選, 對於晶體有較高準確度。至於準確度更高的柔性方法是在剛性後做一個更細化的對接, 因為準確性較高, 一般較常考慮柔性的對接方式。

目前較著名的模擬軟體有 AutoDock、UCSF DOCK 等等, 其中 AutoDock 為小分子對接軟體, 較常被使用。在各樣軟體中彼此有著不同的演算方式, 統計整理目前已經做為應用在模擬的演算法如下[3]:

GA = Genetic algorithms

SM = Shape matching

IC = Incremental construction

MC = Monte Carlo

SA = Simulated annealing

DG = Distance geometry

EP = Evolutionary programming

TS = Tabu search

分子對接模擬能夠協助藥物的開發, 相對的會產生大量的計算。Sally R. Ellingson 及 Sally R. Ellingsony 在 2011 年提出[4]使用 Hadoop 平台執行呼叫 AutoDock, 建構出受體在 Hadoop 中與多分子同時進行對接模擬的架構, 大量提升分子篩選的效率。

本論文的重點是使用 Hadoop 平台與 GPU 建構出高速運算的環境開發分子動力學對接模擬, 透過 Hadoop 及 GPU 的特性來提升模擬受體與配體的對接效率, 主要是在受體中尋找與配體最好的對接位點, 及在上萬種的化合物分子中尋找能夠與目標受體相互對接且有潛力做為藥物的化合物分子, 在 JAVA 及 CUDA (Compute Unified Device Architecture) 上進程式撰寫, 並且進行測試與實作研究。

Hadoop MapReduce[5]主要分為運算與儲存兩大部分, 由 MapReduce 負責運算, HDFS 負責儲存。MapReduce 框架將處理分為 Map 與 Reduce 兩個階段, 建立 Map 與 Reduce 兩個函式控制程式執行流程, 每個階段輸入與輸出都採用 key/value 組合。HDFS 是 Hadoop 檔案系統, 提供存取 Map 與 Reduce 的資料, 最底下的叢集伺服器則是分散式運算中節點的所在地。從 HDFS 讀取資料後, 根據每個 Map 的能力開始分配工作執行, Map 結束後排序合併, 根據相同 key 值傳送到至對應的 Reduce, 最後將結果輸出至 HDFS 中。

GPU 圖形處理器[6]為一種多核心(Multi-core)的平行處理器, 處理單元的數量遠超過 CPU。而 CUDA 為 NVIDIA 所開發的平行運算架構, 利用顯示卡 GPU 強大運算能力來增加運算效能, 平時電腦裡面用 CPU 運算, 但使用 CUDA 後, CPU 變更為控制及排程管理, 透過 CPU 將其他複雜的運算轉到 GPU 上, 因為 GPU 有著更高的運算單元。由於 GPU 內部不支援 I/O 系統, 只能運用 CPU 進行資料搬運的方式移至 GPU 上, 透過 GPU 執行系統, 最終將 GPU 的結果回傳 CPU 內做 I/O 處理。

研究上幾個重點，其一為力場評估函式的研究，在分子之間的評估函式使用凡得瓦力、靜電力及去溶劑來換算自由能數值並評估目前對接狀態的結果。其二為 Hadoop 及 GPU 上使用基因演算法(Genetic Algorithm) 及蒙特卡羅(Monte-Carlo) [7]搜尋演算法來開發一套適用於分子對接模擬的工具，透過 Hadoop MapReduce 框架將資料自動切割，並分散檔案至每個 Map 的節點上，透過 GPU 來進行蒙特卡羅區域搜尋及力場函式計算，最後 Reduce 收集所有 Map 處理過的結果，得到最後搜尋的最佳解。

在下一節中將介紹本論文的研究方法、步驟和結構設計，最後展示本論文的測試效能及實驗結果。

2. 方法

分子對接是藉由將配體分子配置到受體分子的活性位點，根據兩者受到的作用力評估彼此的相互作用，來找出最佳的對接模式。分子對接的概念源自於 1894 年赫爾曼·埃米爾·費歇爾所提出的「鎖與鑰匙(Lock and key)」模組，此模組描述蛋白質和配體會有一定的外型，且兩者之間外型能夠精確的互補時，反應才會發生，宛如是生活中的鎖與鑰匙。所以配體與受體兩者要產生反應，就需符合空間上對接的形狀。

由於模擬須考量所有情況，在一個大空間框架中搜尋最佳的對接位點，相對上會產生大量的運算。模擬需從 PDBQT 檔案取出受體與配體的資訊，再評估兩者原子的能量。將受體中所有原子與配體中所有原子進行一次能量計算，對接的結果轉換成數值以用於評估目前狀態是否為最佳的對接情況，如此可得知目前位置對接狀態的評估能量，最後得到所有原子狀態的能量總和，以此能量總和來評估目前模擬的對接位置是否為良好的對接情況，再移動配體到下一個座標位置，並且重複的進行原子能量運算，如此循環移動、旋轉配體的座標步驟，進行尋找最佳的對接位置，就形成了一個搜尋最佳解的問題。本論文中應用了基因演算法及蒙特卡羅兩種演算法。

2.1 基因演算法(Genetic Algorithm)

在尋找最佳解的問題上，本研究使用基因演算法來進行搜尋，同時考慮搜尋空間中的多個點，因此可以迅速得到全域最佳解，同時也能避免陷入區域最佳解。隨機搜尋最佳解並具有能收斂最佳化的值，根據基因的分裂流程每個世代隨機產生 N 群體，有著交配(crossover)、突變(mutation)、複製(reproduction)等動作，然後評估每個染色體的適應函數，世代結束後再產生大量子代循環動作直到滿足終止條件為止。

基因演算法之效率優劣取決於下列幾個要點：使用者如何去決定基因組、初始族群、適應函數。

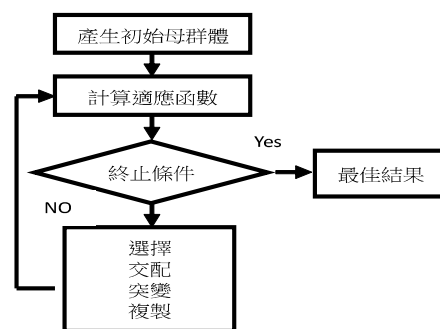


圖 1：基因演算法流程圖

2.2 蒙特卡羅(Monte-Carlo) 搜尋

使用隨機亂數來解決很多計算問題的方法。在一個解空間內利用大量的隨機亂數來找出空間內最接近最佳解的點。Joshua 等人在 2012 年發表[7] 使用 GPU 搭配蒙特卡羅的方法將其應用在模擬上。

2.3 力場評估函式

分子間的作用力取決於現實世界裡的力場函數，論文中參考[8]Autodock 部分的公式，此力場函數(1)是由分子間的凡得瓦力、靜電力及去溶劑所集合。在運算上所需要的參數越多就越為精準，同理就會有越高的計算量。力場函數的固定參數值引用自 Autodock 4.2[9] 參數文件。

$$\Delta G = \Delta G_{vdw} \sum_{i,j} \left\{ \frac{A_{ij} - B_{ij}}{r_{ij}^{12} - r_{ij}^6} \right\} + \Delta G_{elec} \sum_{i,j} \frac{q_i q_j}{\epsilon(r_{ij}) r_{ij}} + \Delta G_{sol} \sum_{i,j} (S_i V_j + S_j V_i) e \left(\frac{-r_{ij}^2}{2\sigma^2} \right) \quad (1)$$

凡得瓦力使用蘭納-瓊斯勢[10] (Lennard-Jones potential function)，蘭納-瓊斯勢本身是一個近似公式，最早從 1931 年提出，是計算化學中用來模擬兩分子間作用能量的一個函數。此能量以兩原子距離做為唯一的變數，包含了兩個參數(2)， ϵ 等於勢能阱的深度(the depth of the potential well)， σ 是互相作用的能量為零時的兩原子距離。在實際應用中， ϵ 、 σ 參數透過已知實驗數據或精確量子計算結果而確定。

$$V(r) = 4\epsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^6 \right] \quad (2)$$

Charles Augustin de Coulomb 在 1785 年制定了平方反比定律說明兩個點之間電荷的作用力[11]。 q_i 、 q_j 分別為部分電荷， r_{ij} 為彼此間的距離，距離跟電荷成反比。 $\epsilon = \epsilon_r * \epsilon_0$ ， ϵ_r 為電容常數， ϵ_0 為真空下的常數 $8.854188 \times 10^{-12} \text{ J}^{-1} \text{ C}^2 \text{ m}^{-1}$ ， ϵ_r 透過已知實驗數據或精確量子計算結果而確定。

$$F = \Delta G_{elec} \sum_{i,j} \frac{q_i q_j}{\epsilon(r_{ij}) r_{ij}} \quad (3)$$

Wesson and Eisenberg 提出[12]去溶劑的方法，他們推測去溶劑的能量與水的表面積改變成比例。透過比較溶劑及表面積可得知鍵結力和非鍵結力。每個類型的原子都有不同的能量，影響著疏水性及極性。因此建構出不同原子的溶劑參數，再透過查表得知。S 為溶劑參數，V 為原子片斷的能量， σ 為高斯的距離常數 3.5 Å，各原子參數透過已知實驗數據或精確量子計算結果而確定。

$$\Delta G_{sol} = W_{desolv} \sum_{i,j} (S_i V_j + S_j V_i) \exp\left(\frac{-r_{ij}^2}{2\sigma^2}\right) \quad (4)$$

透過此力場函數先計算出目前所有原子對接位置的總能量，接著分析先前的對接狀態，比較前後結果並進行修正，推算下一階段的行動並且更新，再反覆的執行力場函數計算，取得所需求的最小能量做為目標的最佳結果。

2.4 系統架構

為實現 Hadoop 及 GPU 建構藥物輔助設計的理想，使用 CPU 與 GPU 運算單元相互輔助建構出分子對接模擬系統，主要分成 Hadoop 與 GPU 兩大部分；Hadoop 使用基因演算法並設定排程將資料搬移至 GPU，透過 GPU 將複雜的運算進行處理並回傳結果給 CPU。

Hadoop 基因演算法將會進行大範圍搜尋，並將搜尋結果傳送給 GPU，而 GPU 則會使用蒙特卡羅演算法進行區域搜尋最佳解。

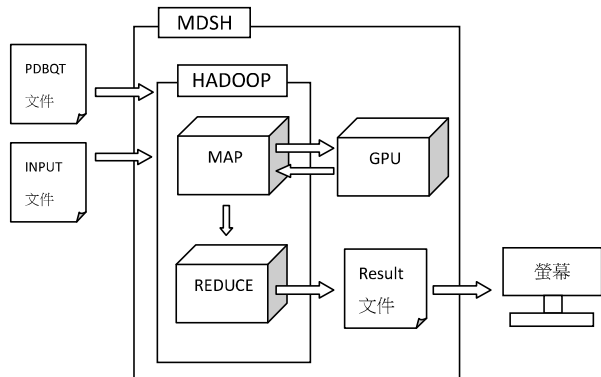


圖 2：MDSH 系統架構圖

Step1 參數初始化

讀取受體以及配體 PDBQT 檔案資料取出原子資訊，將其受體以及配體的座標做初始化。設定 GA 演算法之 MapReduce 的個數及執行次數、GA 演算世代數及執行次數、搜尋的 grid 範圍；母群體(pop)個數(N)將決定初始亂數產生 N 個個體。

Step2 演算法執行

擁有受體以及配體座標後，定義配體座標為基因。執行上，將受體固定在空間中，使用配體以 2Å 距離單

位進行大範圍搜尋，並設定 Map 與 Reduce 函數，限制運算中不得超出 grid 範圍。

每個 Map 函數執行一個 GA 世代，進行世代演化。交配為隨機選擇親代基因以交配的方式產生後代，將會對親代的基因做交換的動作；突變為隨機選擇親代基因並挑選其中的一個或多個基因進行替換。世代演算規則：排序後前段 1/3 不更動，剩下的 2/3 依照機率進行交配或是突變，產生新的個體。

MapReduce 在主程式運行後，map 將資料分割 Map 個數等分，所有 Map 的 key 定為相同數值，分割完成的資料送到 Map 中；value 值為 GA 母群體轉換成的字串。所有 Map 為相同 key 值，key 值相同有助於 Reduce 收集所有 Map 運算之結果。

表 1：個體基因資訊

X 位移	Y 位移	Z 位移	X 角度	Y 角度	Z 角度	自由能
------	------	------	------	------	------	-----

表 2：MapReduce Key Value 設定

Map Input(LongWritable Key, Text Value)
Map Output(LongWritable Key, Text Value)
Reduce Input(LongWritable Key, Iterator<Text> Values)
Reduce Output(LongWritable Key, Text Value)

表 1 表示系統中的個體基因，包含了 XYZ 三維的位移量、XYZ 三軸的旋轉角度及一個自由能。表 2 中為使用 MapReduce 框架所須表明之 Key、Value 之形態，LongWritable 與 Text 皆為 MapReduce 框架之形態，在 java 中可代表為 long 與 String 形態。

Algorithm 1：Map

```

currentGen=0           //世代數
GPU_LocalSearch(pop)   //呼叫 CUDA 進行區域搜尋
RankSelection(pop)     //排序
while(currentGen<Generation_count) {
    NextGeneration(pop) //演化世代
    GPU_LocalSearch(pop)
    RankSelection(pop)
    currentGen++
}

```

Map 演算法步驟如下：

1. 世代數 currentGen 初始化並抓取母群體 pop。
2. 將母群體 pop 傳送至 GPU_LocalSearch 運算後將結果排序。
3. 演化世代迴圈中進行演化運算 NextGeneration，進行交配與突變，並確保在 grid 範圍內。
4. 將新的世代重新傳送至 GPU_LocalSearch 計算後排序，並繼續演化世代迴圈。
5. 判斷 currentGen 是否為設定世代數 Generation_count，或是結果已收斂，決定繼續演化世代迴圈或是結束。

Algorithm 2 : NextGeneration

```

for i = 0 to pop_size{
  Random number           //產生隨機數(0 to 1)
  if number < crossover_rate {
    if number < mutation_rate{ //突變
      do{
        Random parent
        Child=Mutation(parent)
      }while(checkInGrid(Child))
    }
  }
  else{                    //交配
    do{
      Random parent_F,parent_M
      Child=Crossover(parent_F, parent_M)
    }while(checkInGrid(Child))
  }
  pop[i]=Child
}

```

NextGeneration 演算法步驟如下：

1. 產生隨機數決定交配、突變或是不更動。
2. 隨機數小於交配機率時且小於突變機率時，進行突變：隨機選出母群體 pop 之中一個體，隨機選擇一基因產生新值。
3. 隨機數小於交配機率且大於突變機率時，進行交配：隨機選出母群體 pop 之中兩個體，隨機選擇基因進行互換。
4. 將突變或交配的子代更新到母群體 pop 中，不更動則繼續執行迴圈。

產生新的世代後，透過呼叫 GPU_LocalSearch 進行區域的搜尋及力場函式評估適應函數，將後代排序並留下適應函數較高的族群。能量總和越小的配體在現實世界裡的原子狀態越穩定，在藥物設計上，也就有較高的機會能夠與受體結合產生作用。

由於 GPU 擁有高運算元的特性，搭配蒙特卡羅的方法則可以減少各 thread 彼此的相關性，使之個別運作，讓大量的 threads 在最短時間各自取得結果，再透過整合的方式，取出本次最佳的解。

表 3：GPU 參數設定

GPU Input(int BlockCount, double Distance_X, double Distance_Y, double Distance_Z, double Rotate_X, double Rotate_Y, double Rotate_Z)
GPU Output(Text value)

表 3 中所需給予的參數主要是 BlockCount，再來是六個 double 一組的座標參數。根據 BlockCount 來決定需要輸入座標組的數量，BlockCount*(6*double)。每個 block 運算一組座標，透過 Map 呼叫給予所需的 BlockCount，可以增加使用系統的彈性。

Algorithm 3 : GPU_LocalSearch

```

getBlockcount()           //取得 block 數量
getdataXYZ()              //取得座標資訊
score_data = GPU_MCselect() //local search
for i=1 to Blockcount
  在 score_data[i]搜尋 best_score_data
return best_score_data

```

GPU_LocalSearch 演算法步驟如下：

1. 從 INPUT 檔中抓取所需的設定參數。
2. 接收 Map 傳輸的參數。
3. 根據設定中的 blocks、threads 準備 GPU 環境。
4. 執行 GPU 區域搜尋，將一組座標資訊帶入一個 block 內，使用蒙特卡羅方法，運用大量的 threads 進行隨機式搜尋。
5. 接收從 GPU 回傳回來的結果，並從 threads 中找出最佳解，回傳本次結果給 Map。

Algorithm 4 : GPU_MCselect

```

__shared__ double BlockEnergy[THREAD_SIZE];
__shared__ int bestID;           //block 最佳解 id
const int tid = threadIdx.x;     //thread ID
const int bid = blockIdx.x;     //block ID
VALUE valueID;                  //儲存 XYZ 六個 double 型態的 data

```

```

while( check() ) {              //在 grid 內產生亂數 value
  以 0.375 Å 為單位在 2 Å 的範圍內 Random 三個參數
  Distance_X, Distance_Y, Distance_Z, 以及以 1° 為單位
  在 360° 的範圍內 Random 三個旋轉角度參數 Rotate_X,
  Rotate_Y, Rotate_Z, 將其存入 valueID
}

```

```

BlockEnergy[tid]=Scoreing() //評估函式

```

```

__syncthreads()                //block 等待同步
if tid==0 {                     //找最好結果的 thread
  bestID=0
  for i=1 to THREAD_SIZE {
    if BlockEnergy [bestID]>BlockEnergy [i]
      bestID=i
  }
}

```

```

__syncthreads()
if tid==bestID {                //最好結果的 thread 進行存取
  Bestvalue = valueID
  Bestenergy = BlockEnergy [tid]
}
return Bestvalue, Bestenergy

```

GPU_MCselect 演算法步驟如下：

1. 每個 block 進行區域的搜尋。在 thread 中各自 Random 參數，且確認 Random 的參數不超過 grid。

2. 將 thread 的 Random 搜尋結果進行力場函式的評估 Scoring，且將評分結果儲存在 shared memory 中。
3. 等待所有 threads 運算結束後，在 shared memory 中尋找本次最佳解的 thread ID。
4. 再將擁有最佳解的 thread 執行最終結果的資料儲存。
5. 將結果回傳給 CPU 做統整。

運算適應函數耗費較多的時間，GPU 擁有較多的運算單元可以進行區域的搜尋以及計算，所以將基因資訊搬移至 GPU 上執行，在接收到 XYZ 的參數，GPU 使用平行架構及蒙特卡羅方式讓每組資料同步搜尋，在 XYZ 軸上做移動及旋轉，以 $0.375 \text{ \AA}$ 為單位在 $2 \text{ \AA}$ 的範圍對鄰近區域進行 1024 次的探索，探索後再回傳給 Map 最佳的結果做為基因。

Reduce 函數則根據 key 值收集所有 Map 的 value，將所有 Map 運算完的個體排序並將資料寫入 HDFS 中。

Step3 結果輸出

Reduce 將 Map 的結果做整合排名，把前 10 名的結果做 PDBQT 檔案輸出，透過介面顯示工具分析觀察受體以及配體的對接情況。

3. 結果

系統測試環境為 4 節點叢集電腦(CPU: Intel i7 - 4 cores 2.8 GHz, 記憶體: 8G DDR3, 顯示卡: NVIDIA Tesla C2050), 作業系統: Centos 6.3 64bit, Hadoop: 1.0.4, CUDA: 5.0。

本系統測試將與 Autodock(版本 4.2)比較其結果，Autodock 執行環境使用其中一個節點執行。Autodock 設定 GA 參數則為同樣數值。母群體數: 120, 演化世代數: 50, 交配率: 0.8, 突變率: 0.02, Map 個數: 4。

在 CUDA 中設定每個個體將會搜尋 1024 次，並回傳最好的結果，故最大搜尋次數為 $1024(\text{CUDA}) \times 120(\text{母群體數}) \times 50(\text{世代數}) = 614.4 \text{ 萬次}$ ；Autodock 則為 $27000(\text{世代數}) \times 120(\text{母群體數}) = 324 \text{ 萬次}$ 。MDSH 計算量較 Autodock 多，找尋更多可能的結果。

RMSD(Root Mean Square Deviation)[2]值為分子對接軟體中常用的評估方法，可以算出其目標構象(conformation)與結果構象的均方根誤差，辨別模擬結果與實際結果的是否相同。

表 4：由參考文獻[13, 14]選出 17 筆測試資料(PDB ID)

1abf	1c83	1lst	1tng	1tnh	1tni
1tnj	1tnk	1tnl	1xie	3ptb	5abp
1fen	1fkg	1qcf	2cmd	2gbp	

表 4 中所使用的測試資料為參考文獻[13, 14]中選出，已經由文獻驗證可使用之資料，該文獻研究內容在於探討 Protein-Ligand Interface，故較具有研究性質。

系統著重於 Autodock RMSD 值小誤差情況下其運算時間的比較。以下測試數據為對接結束後，依照能量排序前 10 名，取最小的 RMSD 數值。

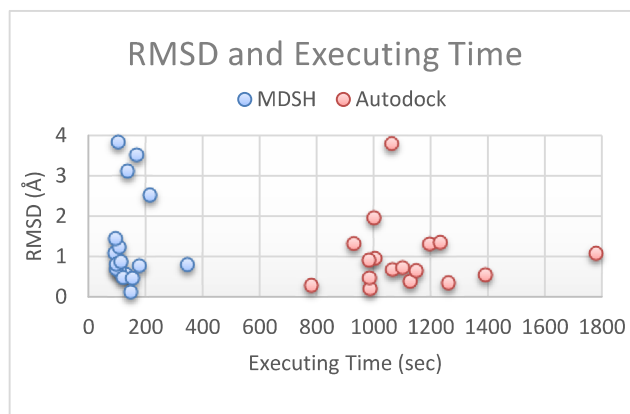


圖 3：X 軸為時間，Y 軸為 RMSD 值

由圖 3 中可見 MDSH 與 Autodock 之 RMSD 值群聚於 2 以下區間，只有少數幾個超出 2，甚至有些結果比 Autodock 好，但 Autodock 執行時間過長，而 MDSH 在時間上大幅度縮短，顯著表現 Hadoop 平台與 CUDA 結合之 MDSH 系統效益性，提供高效率的計算速度與準確度。

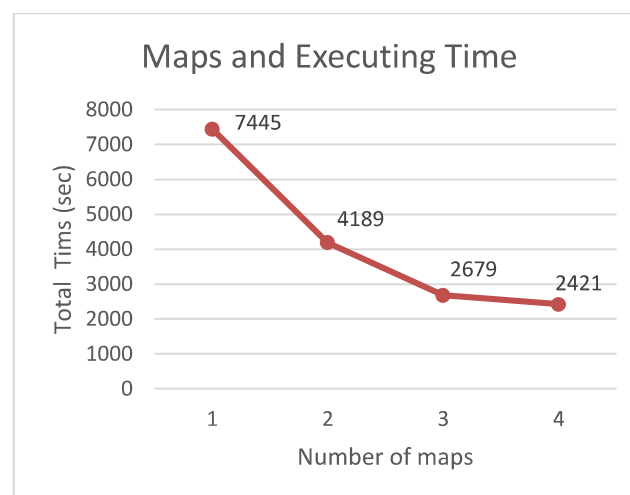


圖 4：X 軸為 Map 個數，Y 軸為 17 筆資料加總執行時間

由圖 4 中可見，在 Map 數提高的情況下，其運算時間會降低，執行效率變高。

4. 結論

在測試結果中，MDSH RMSD 值已符合 Autodock 之準確度甚至有較佳結果出現，且在運行效率上高於 Autodock。

由於本系統實驗叢集只有 4 個節點，在 Map 與執行時間上的測試結果數據上無法顯著展示，未來希望能將系統建置於更多的叢集節點中，提高執行效率。

模擬需要大量的運算，Hadoop 平台及 GPU 的多核心處理器提供了這種需求，使用分散式平行化系統去運算處理，在運算效率上有顯著的提升。

市面上常用模擬對接軟體的效能還有提升的空間，所以改進模擬內部的演算法，讓受體和配體有較高的機率篩選出可以作用的對接組合模式。可以減少執行等待的時間並提升速度，再透過改良內部的演算法，對於輔助藥物設計有所幫助。

5. 致謝

本論文由國科會計畫編號：NSC101-2221-E-320-007 支持。

6. 參考文獻

- [1] S. Cosconati, S. Forli, A. L. Perryman, R. Harris, D. S. Goodsell, and A. J. Olson, "Virtual Screening with AutoDock: Theory and Practice," *Expert Opin Drug Discov*, vol. 5, pp. 597-607, Jun 1 2010.
- [2] D. S. Goodsell, G. M. Morris, and A. J. Olson, "Automated docking of flexible ligands: applications of AutoDock," *J Mol Recognit*, vol. 9, pp. 1-5, Jan-Feb 1996.
- [3] R. Dias and W. F. de Azevedo, Jr., "Molecular docking algorithms," *Curr Drug Targets*, vol. 9, pp. 1040-7, Dec 2008.
- [4] S. R. Ellingson and J. Baudry, "High-throughput virtual molecular docking: Hadoop implementation of AutoDock4 on a private cloud," presented at the Proceedings of the second international workshop on Emerging computational methods for the life sciences, San Jose, California, USA, 2011.
- [5] A. S. Dino Keco, "Parallelization of genetic algorithms using Hadoop Map/Reduce," *SOUTHEAST EUROPE JOURNAL OF SOFT COMPUTING*, vol. 1, p. 5, 2012.
- [6] S. Ryoo, C. I. Rodrigues, S. S. Bagsorkhi, S. S. Stone, D. B. Kirk, and W.-m. W. Hwu, "Optimization principles and application performance evaluation of a multithreaded GPU using CUDA," presented at the Proceedings of the 13th ACM SIGPLAN Symposium on Principles and practice of parallel programming, Salt Lake City, UT, USA, 2008.
- [7] A. A. Joshua, J. Eric, L. G. Thomas, E. Michael, and C. G. Sharon, "Massively parallel Monte Carlo for many-particle simulations on GPUs," *CORD Conference Proceedings*, 2012.
- [8] G. M. Morris, D. S. Goodsell, R. S. Halliday, R. Huey, W. E. Hart, R. K. Belew, and A. J. Olson, "Automated docking using a Lamarckian genetic algorithm and an empirical binding free energy function," *J Comput Chem*, vol. 19, pp. 1639-1662, 1998.
- [9] G. M. Morris, D. S. Goodsell, M. E. Pique, W. L. Lindstrom, R. Huey, S. Forli, W. E. Hart, S. Halliday, R. Belew, and A. J. Olson, "User Guide : AutoDock Version 4.2 : Automated Docking of Flexible Ligands to Flexible Receptors," 2010.
- [10] J. E. Lennard-Jones, "Cohesion," *Proceedings of the Physical Society*, vol. 43, p. 461, 1931.
- [11] Y. Duan, C. Wu, S. Chowdhury, M. C. Lee, G. Xiong, W. Zhang, R. Yang, P. Cieplak, R. Luo, T. Lee, J. Caldwell, J. Wang, and P. Kollman, "A point-charge force field for molecular mechanics simulations of proteins based on condensed-phase quantum mechanical calculations," *J Comput Chem*, vol. 24, pp. 1999-2012, Dec 2003.
- [12] R. Huey, G. M. Morris, A. J. Olson, and D. S. Goodsell, "A semiempirical free energy force field with charge-based desolvation," *J Comput Chem*, vol. 28, pp. 1145-1152, 2007.
- [13] B. Kramer, M. Rarey, and T. Lengauer, "Evaluation of the FLEXX incremental construction algorithm for protein-ligand docking," *Proteins*, vol. 37, pp. 228-41, Nov 1 1999.
- [14] S. K. Panigrahi and G. R. Desiraju, "Strong and weak hydrogen bonds in the protein-ligand interface," *Proteins*, vol. 67, pp. 128-141, Apr 1 2007.